

1. Situation Analysis

Your system must continuously:

1. Collect global news
2. Process articles
3. Detect geopolitical events
4. Map events to sectors and assets
5. Send them to the prediction engine

This pipeline must run **automatically and reliably**.

2. Core Insight

The key innovation is **transforming unstructured news into structured geopolitical events**.

Raw news:

```
"China restricts gallium exports citing national security concerns."
```

Structured event:

```
Event Type: Trade Policy
Country: China
Sector: Semiconductors
Affected Assets: NVDA, TSM, AMD
Impact: Negative
Confidence: 0.71
```

Your platform's moat becomes the **Event → Market Impact dataset**.

3. Strategic Architecture

News Intelligence Pipeline:

```
News Sources
↓
Ingestion Service
↓
Raw Article Database
↓
NLP Event Extraction
↓
Event Classification
↓
```

Market Mapping
↓
Prediction Engine
↓
Dashboard

4. News Ingestion Service (Step 2)

This service **continuously collects global news**.

Data Sources

Start with reliable APIs:

NewsAPI
GDELT
EventRegistry
Mediastack
Reuters RSS
AP RSS
Al Jazeera RSS

RSS is very important because:

- free
- reliable
- large coverage

5. News Ingestion Architecture

Pipeline:

Scheduler
↓
Fetch News
↓
Normalize Articles
↓
Deduplicate
↓
Store Articles
↓
Send to NLP Pipeline

6. News Ingestion Tech Stack

Backend worker:

Python
FastAPI
Celery
Redis
PostgreSQL

Scheduling:

Apache Airflow
or
Celery Beat

7. Example News Ingestion Code

Basic ingestion worker.

```
import requests
import psycopg2

API_KEY = "newsapi_key"

url = f"https://newsapi.org/v2/top-
headlines?category=business&apiKey={API_KEY}"

response = requests.get(url).json()

articles = response["articles"]

for article in articles:
    title = article["title"]
    content = article["content"]
    source = article["source"]["name"]
    url = article["url"]

    insert_query = """
    INSERT INTO news_articles (title, content, source, url)
    VALUES (%s, %s, %s, %s)
    """

    cursor.execute(insert_query, (title, content, source, url))
```

8. Deduplication System

News often appears across multiple outlets.

Solution:

Use **content hashing**.

Example:

```
SHA256(title + description)
```

If hash already exists:

```
skip article
```

9. Article Processing Queue

Once articles are stored:

They go into **Kafka topic or task queue**.

```
news_articles
↓
NLP queue
↓
event extraction
```

Tools:

```
Kafka
Celery
RabbitMQ
```

10. Event Extraction NLP Model (Step 3)

This model converts articles into **structured events**.

Pipeline:

```
Article
↓
NER (Entity Detection)
↓
Event Classification
↓
Sector Mapping
↓
Asset Mapping
```

11. Named Entity Recognition

Extract key entities.

Example:

```
China
Taiwan
TSMC
Nvidia
```

Semiconductors

Libraries:

spaCy
HuggingFace transformers

Example:

```
import spacy

nlp = spacy.load("en_core_web_trf")

doc = nlp(article_text)

for ent in doc.ents:
    print(ent.text, ent.label_)
```

Outputs:

China → GPE
TSMC → ORG
Nvidia → ORG

12. Event Classification Model

Classifies event types.

Example categories:

Conflict
Sanctions
Trade policy
Energy disruption
Regulation
Elections
Economic data

Model options:

BERT classifier
RoBERTa
DistilBERT

Example training input:

Text:
"China bans gallium exports."

Label:
Trade Policy

13. Sentiment Analysis

Detects market sentiment.

Example:

negative sentiment → bearish
positive sentiment → bullish

Model:

FinBERT

14. Sector Mapping

Map events to sectors.

Example:

Event:
China restricts chip exports

Sector:
Semiconductors

Sector taxonomy:

Energy
Technology
Defense
Agriculture
Metals
Shipping
Banking

15. Asset Mapping

Map sectors to assets.

Example table:

Semiconductors
→ NVDA
→ AMD
→ TSM

This mapping can initially be **rule-based**.

Example:

```
if sector == "Energy":  
    assets = ["XOM", "CVX", "BP"]
```

Later you can train models.

16. Example Event Extraction Output

Final structured event:

```
{  
  "title": "China restricts gallium exports",  
  "event_type": "Trade Policy",  
  "country": "China",  
  "sector": "Semiconductors",  
  "affected_assets": ["NVDA", "AMD", "TSM"],  
  "impact": "negative",  
  "confidence": 0.74  
}
```

This is saved to:

```
events  
event_impacts
```

17. Performance Requirements

Pipeline must support:

```
1000+ articles per hour  
real-time updates
```

Latency target:

```
< 30 seconds per event
```

18. Scaling Strategy

For scale:

```
Kafka partitions  
NLP workers  
GPU inference
```

Model serving:

19. Risk Analysis

Risk 1 — False event detection

Mitigation:

```
confidence threshold
human verification initially
```

Risk 2 — News bias

Mitigation:

```
multi-source aggregation
```

Risk 3 — Data overload

Mitigation:

```
filter by geopolitical keywords
```

20. Tactical Execution Plan

Week 1

```
Set up news ingestion
store articles
```

Week 2

```
NER extraction
entity database
```

Week 3

```
event classifier
```

Week 4

```
sector mapping
asset mapping
```